Lab 43: Valid JSON, wrong refund: add business rules

Topic 4: Prompt Engineering & Structured Output · Task 4.4: Implement validation, retry, and feedback loops for extraction quality

Runs on Simulator 1: Customer Support Resolution Agent · Run in class

Source: SkillCertPro practice exam Part 1, Q4 (unofficial third-party question)

Commit to an answer first

During pilot testing, the refund-recommendation step now always produces parsable structured data, with required fields present and numeric types correct. The remaining bad cases involve merchandise credits placed in the shipping_refund field, or itemized adjustments that do not match the requested refund total. What change should this team make next?

A. Make every refund field required in the schema and reject nullable values during extraction.

B. Add downstream business-rule validation before refund execution and feed specific failures back for correction.

C. Rely on strict tool inputs to guarantee the refund recommendation is operationally correct.

D. Switch to plain JSON instructions and remove tool schemas from the extraction step.

Simulate the solution

```
# In the browser: open Simulator 1, then Lab 43
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 43

# In a terminal, from simulators/simulator-1-customer-support/
uv run backend_tools.py --demo
uv run support_agent.py --case CS-13
uv run support_agent.py --case CS-13 --no-validation --tag novalidation
```

What you should see

Cash refund is $105. The $8 shipping stays store credit.

evidence/runs/CS-13.json: recommend_refund is validated (or rejected with named failures and corrected) before process_refund issues exactly $105; with --no-validation nothing checks the recommendation. evidence/triage-triage_template.json shows the same rule set applied to archived transcripts.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

The answer

B. Validate invariants before execution and feed failures back.

A schema controls shape , not meaning . Structured outputs and strict tool use remove malformed JSON, missing required fields and wrong types, but they cannot know that a merchandise credit landed in the shipping field or that itemised lines should add up to the total. With financial consequences, keep the schema and add validation of business invariants before execution, then send the specific failures back for correction or route the case.

Why the others fail

A. The fields are already present. Making more fields required and non-nullable can push the model to fabricate values, and it still checks nothing about meaning.

C. Strict tool use guarantees schema conformance, not operational correctness.

D. Throws away the guarantees that already fixed the format errors, a step backwards.

Check yourself

- the learner can name one check the schema cannot express
- a failed record is retried with the specific error text appended

Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.